NammaSpace: Walkable Digital Twins of Indoor Spaces from a Single Phone Video

Team Matrix · Chepuri Dathatreya · IIT Techfest 2026, NammaSpace challenge, Round 1: Technical Abstract

Project page, report, demo video: https://namma-space-documentation.pages.dev/ · **Live viewer (Room):** https://namma-space.pages.dev/room/ · **Code:** https://github.com/chepuridathathreya/Namma-Space

1. Summary

NammaSpace rebuilds a real indoor space from one handheld phone video and lets a person walk through it in an ordinary web browser, with no scanner, no depth sensor and no hand modelling. The video becomes a **3D Gaussian splat** [1], a set of about half a million soft, coloured, translucent 3D Gaussians fitted to the real frames, so the room keeps its real colour, reflections and soft edges. A splat has no solid surfaces, so we keep appearance and physics in two separate representations: the splat draws the room, and a **10 cm collision grid** built from the scan itself decides where the viewer can stand. The output is a static website of about 8 MB per scene that needs no server-side code.

We built the pipeline and the capture procedure (SOP) together, because most failures we met were capture faults, not software faults. The best scene, the Room (613 frames), reaches a held-out PSNR of 27.14 dB, SSIM 0.913 and LPIPS 0.191, after 819 s of training on one GPU. Three scenes are live and walkable today.

2. System architecture

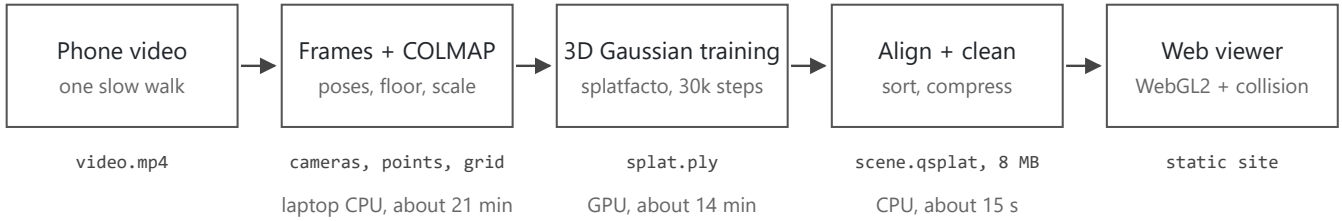


Figure 1. Five stages. Each arrow is a file. Timings are for the Room scene (613 frames).

The work is split into three programs with clear hand-offs. **Prepare** (CPU) turns the video into sharp frames, camera poses, a metric world frame and a first collision grid. **Train** (GPU, 16 GB or more) fits the splat with nerfstudio's *splatfacto* [3] and the gsplat library [4]. **Finish** (CPU, seconds) aligns the splat to the world frame, cleans it, compresses it and builds the website. One script, `run_all.py`, runs all three on one machine. Our own setup runs Prepare and Finish on a laptop and Train on a shared GPU PC, because the laptop GPU has 6 GB.

3. Pipeline design

3.1 Frame selection

A video has far more frames than reconstruction needs, and hand shake blurs some. Every frame is scored for sharpness (variance of the Laplacian on a reduced copy), the video is cut into half-second windows, and the sharpest frame of each window is kept. A window with no frame above the sharpness threshold is reported. A colour check stops the pipeline if frames are grey: an early version scored sharpness on a greyscale copy and accidentally saved that copy, which produced grey scenes.

3.2 Camera poses

COLMAP [2] runs feature extraction, sequential matching (each frame against its neighbours in time, which suits video) and mapping. A health check counts the placed frames, keeps the largest model, warns below 80% placed or when the walk breaks into two large pieces, and stops below 40%. Poses are final once solved. This is the slowest step on a laptop: about 21 minutes for the Room, 87 minutes for a 7-minute capture (883 frames), mostly CPU bundle adjustment.

3.3 Floor, scale and the world frame

COLMAP coordinates have no up direction and no units. A plane is fitted to the lowest points under the walking path, which gives the up direction and sets the floor to height zero. Scale is set by assuming the phone was 1.4 m above the floor. This is an assumption, not a measurement: we have not checked it against a tape measure, so every dimension in

metres in this abstract is approximate and relative to that assumption. The result, `world.json`, maps COLMAP coordinates into one metric frame with Y up. Sanity checks print where known objects land (a bed surface should be 0.45 to 0.55 m, a ceiling 2.5 to 3.1 m). The fixed phone height is an assumption, not a measurement, and is listed under limitations.

3.4 Splat training

30,000 steps of splatfacto at full image size, every frame used except those held out for scoring. The Room took 819 s on an NVIDIA RTX PRO 6000 Blackwell GPU with a peak of 16,251 MiB measured over the whole card. The export has 541,337 Gaussians after 7,444 almost-transparent ones are dropped.

3.5 Alignment, cleaning and compression

The finishing step reverses nerfstudio's re-centring and applies `world.json`, so the splat lands in the same metric frame as the collision grid. It verifies the alignment by measuring the distance from the Gaussians to the real COLMAP points and refuses to publish if the median exceeds 0.4 m. Cleaning removes almost-transparent Gaussians, absurdly large ones, and floaters more than 1.2 m from anything the cameras saw. Glossy floors and wardrobes cannot be modelled as mirrors, so training invents ghost copies below the floor; everything more than 12 cm under the floor, and anything above 3.3 m, is deleted. The rest is sorted most-important-first (opacity times size), so any file prefix is the best subset, and packed into our own `.qsplat` format: 17 bytes per Gaussian, gzip-compressed, about 8 MB for the Room.

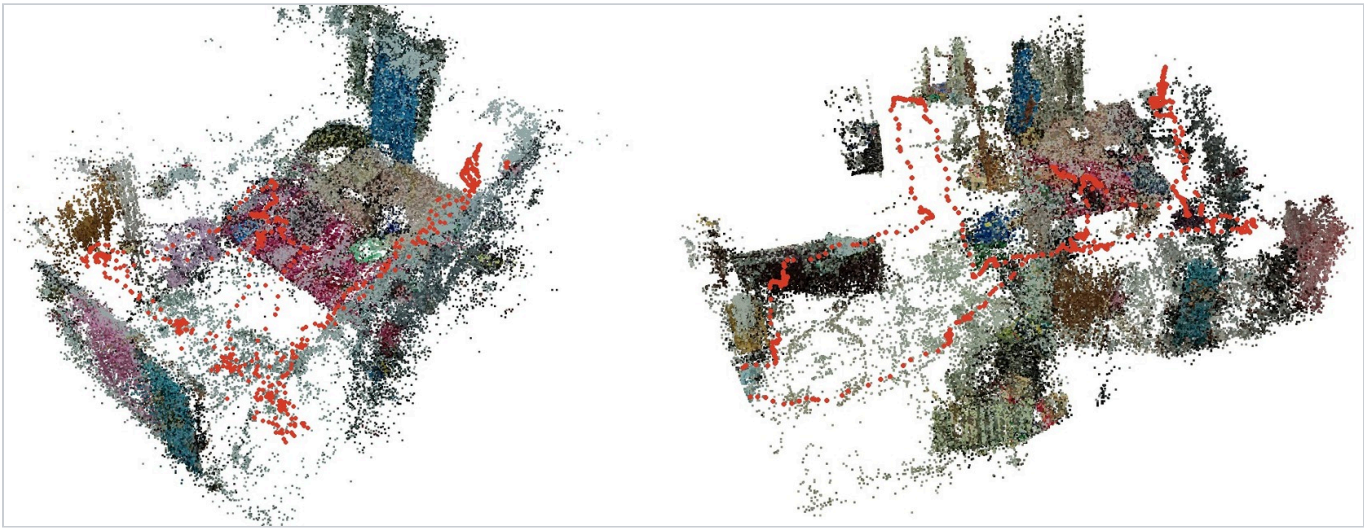


Figure 2. Sparse COLMAP reconstructions with the camera path in red: the Room (left) and a longer hallway and bedroom capture (right). Ceiling points are removed so the interior is visible.

4. SOP construction

The capture procedure was written from what we did for the Room, our best result, and from what went wrong earlier. It does not describe a method we did not use. The full text is in the repository (`docs/CAPTURE_SOP.md`) and on the project page.

4.1 Equipment and fixed settings

Item	Setting (Room scene)
Phone and mode	POCO X6, Pro (manual) mode for both video and photos
Lens	0.6x ultra-wide, chosen once and used for the whole capture; covers more wall in a small space
Focus, shutter, ISO	Manual focus, 1/100 s, ISO 800
White balance, EV	5000 K fixed, EV 0
Video	1920 x 1080, 30 fps, landscape

Fixed settings matter because automatic exposure and white balance make the same wall change brightness from frame to frame, which makes matching harder and gives the trained splat visible brightness jumps.

4.2 Before filming and the walk

- Aim for even lighting and close curtains that throw bright patches. Keep people and pets out, since moving things leave ghosts. Wipe the lens. Cover mirrors (we used a towel).

- Hold the phone at chest or eye height, landscape, tilted slightly down. Walk **one continuous lap** at about half a metre per second, turn slowly, finish where you started. If the ceiling is wanted, tilt up slowly during the lap and back down.
- **Never walk a second lap at another height or angle.** In our first hallway capture a second, raised lap made the same wall appear twice: bent doors and a double window.

4.3 Extra photos for corners and ceiling

A walk shows corners and the top of the walls only briefly. After the video we took still photos from the same eye height with the same lens and Pro settings: three to four per corner, a small step apart, tilting up for the ceiling and down at corners, each showing something the video also shows so COLMAP can match them. The Room used 26 photos, all registered. They are placed in their own folder and added to the frame set automatically.

4.4 Checks and known failure cases

The pipeline stops on a failed colour check, a frame-alignment mismatch or a poor camera model. For the Room: 587 sharpest-frame picks plus 26 photos, 613 of 613 placed, 135,780 points. Failure cases we recorded and the rule each produced:

Problem	Cause	Rule
Doubled walls, bent doors	Second lap at another angle	One lap, one height
Black ceiling (Early Bedroom)	Plain white ceiling never filmed	Tilt up during the walk, add ceiling photos
Ghost copies below floor	Splat copies reflections on glossy floors, wardrobes, mirrors	Cover mirrors; pipeline deletes ghosts under the floor
Fewer frames placed	Fast or broken walk	Slow, continuous walk
Rough corners and ceilings	Barely covered	Extra photos

Not recorded or not yet tested: lighting was not controlled or measured; capture time (roughly 8 to 10 minutes for video plus photos) is an estimate, not a timed result; a 4K re-capture is still open. We have not run an experiment that isolates the effect of the extra photos.

5. Collision and the web viewer

5.1 Collision grid

Cells are 10 cm. A cell is an obstacle when at least three points lie in it between 0.15 m and 1.7 m above the floor. Free space comes from cells near the walked path and floor cells seen near it, and anything within 20 cm of the path cannot be an obstacle. A 25 cm margin (the player radius) is added around obstacles. The trained splat is a second, independent source of evidence: a map built from the cleaned splat is combined with the camera-path map, only the connected region containing the walked path is kept, and a cell stays blocked where the splat map sees a solid obstacle. The step stops if the walked path or start point would end up blocked. The grid is derived automatically from reconstruction data (camera path, sparse points and splat evidence). It has not been validated against measured wall, furniture or doorway positions, so it should be read as a believable, consistent walking boundary, not as surveyed geometry. On the Room this removed 17 of 430 walkable cells while the walked path stayed 100% walkable.

5.2 Viewer

The viewer is plain static files with no build step. It decodes the `.qsplat`, draws the Gaussians with our own WebGL2 renderer and sorts them back to front continuously so translucent blobs blend correctly. Movement is first-person: W A S D, mouse look, run and jump. Each frame it tries to move and, if the target cell is blocked, slides along the wall. A live map drawn from the collision grid shows walkable floor, obstacles, the position and the viewing cone. Because the file is sorted best-first, buttons can draw 250k, 500k or all Gaussians.

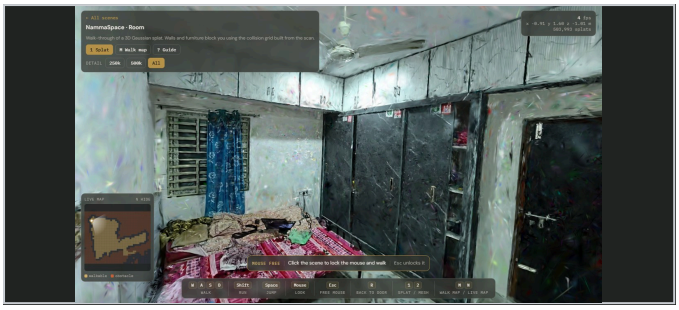


Figure 3. The viewer in the Room: live map at the bottom left, grouped controls bar.

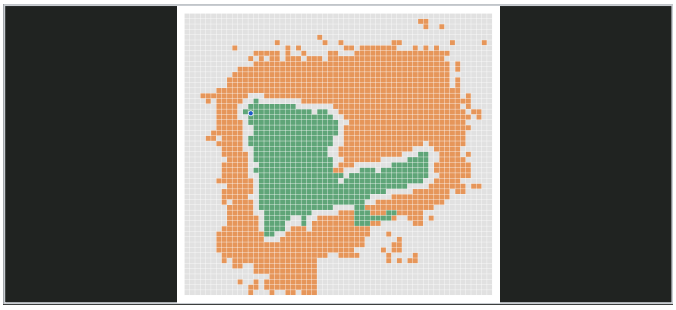


Figure 4. Room collision grid: walkable (green), obstacle (orange), blocked margin (grey).

5.3 Performance

Frame rate depends on how many Gaussians are drawn and which GPU the browser uses. In informal tests we saw 24 to 38 fps with all Gaussians on a laptop's integrated GPU and 170 to 180 fps on the RTX 3060 laptop GPU. These are single-machine observations with all Gaussians drawn, not a standardized benchmark (browser version, resolution, test length and camera path were not controlled), so they are not a guarantee for other hardware. A web page cannot choose the GPU, so the viewer measures its frame rate about 2.5 s after loading and, below 35 fps, steps down through fewer Gaussians and lower render resolution and tells the user.

6. Results

Scene	Frames	Gaussians	PSNR	SSIM	LPIPS	Training	Walkable floor
Room	613	541,337	27.14 dB	0.913	0.191	819 s	4.3 m^2
Hallway and Bedroom (v2)	868	650,654	26.46 dB	0.874	0.244	995 s	15.8 m^2
Early Bedroom	n/a	700,000 shown	n/a	n/a	n/a	n/a	8.1 m^2

PSNR and SSIM: higher is better. LPIPS: lower is better. Scores are on frames held out from training (hallway: every eighth frame, 109 of 868). The Early Bedroom predates our evaluation step. The Room is a different scene from the hallway, so its higher score may come from the capture method or simply from an easier room; we do not claim a controlled improvement. Within the hallway family a first run scored 23.56 dB on 709 frames and the improved run 26.46 dB on 868 frames, but frame count, pipeline fixes and capture conditions all changed between them, so that gain is not attributed to any single cause.

7. Problems met and what changed

Each scene exposed a fault that changed the pipeline. Some are fixed, some only reduced; the full list is in section 10 of the online technical report (https://namma-space-documentation.pages.dev/report/), not in this PDF.

- **Capture:** black ceiling, doubled walls from a second lap, ghost reflections. Produced the one-lap rule, the extra-photo step and the under-floor cleanup.
- **Software:** grey frames (now checked at the start of both steps), a fake splat made from raw COLMAP points (removed; a splat must come from real training), two scenes on one site (now one video is one scene), a wrong header that made the compressed file unreadable, and viewer faults (black screen on a scene without a mesh, a late depth-sort result drawing a scrambled frame, and collision working only near the walked path).
- **Compute:** a training crash we first blamed on the graphics driver was the laptop running out of RAM, so training runs only on a GPU machine. The Blackwell GPU needs CUDA 12.8 builds, so we built a matching environment.

8. Reproducibility, limits and next steps

Reproducibility. The repository holds the three scenes, the pipeline, a requirement checker and a bundled sample in `test/`. `python run_all.py --check` lists missing requirements, `--test` runs the sample with a short training, and `--video my_walkthrough.mp4` runs a full capture. Requirements: Linux or WSL2, NVIDIA GPU with 16 GB or more, 20 GB disk, ffmpeg, COLMAP 3.9 or newer, nerfstudio with gsplat. **Tested:** preparation and finishing on Windows, training on Linux in our two-machine setup, the requirement check and finishing on the bundled sample. **Not yet tested:** the one-machine training run on a separate clean GPU machine.

Limitations. Ceilings and unseen areas are rough or dark. Collision is two-dimensional and does not model stairs or overhangs. Scale is assumed from a 1.4 m phone height. A fast or broken walk reduces the frames COLMAP can place.

Training needs about 16 GB of GPU memory.

Next. Points of interest placed on the same grid the live map reads, and a navigation method that keeps routes inside walkable space (Round 2); faster and more robust reconstruction with a learned model, tested against the current pipeline on the same held-out frames; filling unseen areas; an augmented reality view; and stronger collision where the two sources of evidence disagree.

References

[1] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM TOG (SIGGRAPH), 2023. [2] J. L. Schönberger, J.-M. Frahm. Structure-from-Motion Revisited. CVPR, 2016. [3] M. Tancik et al. Nerfstudio: A Modular Framework for Neural Radiance Field Development. ACM SIGGRAPH, 2023. [4] V. Ye et al. gsplat: An Open-Source Library for Gaussian Splatting. arXiv:2409.06765, 2024.